

P2958R0

typeof and **typeof_unqual** for C++

Published Proposal, 2023-08-05

This version:

https://thephd.dev/_vendor/future_cxx/papers/d2958.html

Author:

JeanHeyd Meneide (<https://thephd.dev>)

Audience:

EWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Target:

C++26

Latest:

https://thephd.dev/_vendor/future_cxx/papers/d2958.html

Abstract

The long-anticipated type features from C have been integrated into C23, and should be ported to WG21 for harmony.

Table of Contents

1 Revision History

1.1 Revision 0 - August 5th, 2023

2 Introduction and Motivation

3 Specification

3.1 Language Wording

- 3.1.1 Add to Table 5: Keywords ([`tab:lex.key`]) 2 new keywords
- 3.1.2 Modify "Decltype specifiers" [dcl.type.decltype]
- 3.1.3 Modify `decltype` mention in Template Deduction's General Clause [temp.deduct.general] to include `typeof` and `typeof_unqual`

References

Informative References

§ 1. Revision History

§ 1.1. Revision 0 - August 5th, 2023

— Initial Release! 🎉

§ 2. Introduction and Motivation

In the oldest proposals for `decltype( ... )`, the proposal authors noted how the construct would produce a reference under common situations involving declarations and usage of C code inside of it. This was part of a sincere argument to leave `typeof` alone as a keyword and instead pivot to the `decltype` keyword instead, with some authors directly pointing out serious flaws with trying to take the `typeof` keyword from the existing vendor extension and C implementation space [n2343] [n1978].

The intention was that keyword differed enough from the existing practice from C and C++ compilers with the `typeof` and `__typeof__` extensions that they should leave it alone, and let C standardize it at its own pace. C23 added `typeof` and, at the request of users working on the Linux Kernel and within the GCC bug tracker, `typeof_unqual` to solve the distinct issues with C [n2927]. Furthermore, it keeps with the existing practice that allows both expressions and type names to go into the `typeof` operator in a way that `decltype` chose not to.

Critically, this means that `typeof` and `__typeof__` cannot be fully approximated with `decltype` alone, or even a macro such as `#define typeof(...)`

```
::std::remove_reference_t<decltype(__VA_ARGS__)>.
```

C and C++ implementations the `typeof` extension can already take a type specifier (or, more specifically for the case of C++ grammar, a *type-id*). This is exceptionally important for `typeof` usage in macros: often times, rather than inventing an expression with which to provide the right type to a macro, it can be expedient to just pass a type in directly where possible to control the eventual cast or type inside of particularly

complex macros. This is done a handful of times in C world. This is not as important for C++, but (some) macros tend to live in a shared world for C and C++.

Some thought was given whether for this proposal to allow `decltype( type-id )` in the grammar as a type-based pass-through. An earlier revisions of [n1978] (N1607, to be precise) states, it was left untouched so to let it potentially be used as a "meta"-type value. That paper even casually references the idea of doing `decltype( SomeType ).is_reference( )`. We believe it is highly unlikely this will ever happen given the direction reflection has gone in for its facilities (e.g. preferring a sigil/operator or the keyword `reflexpr`). Furthermore, the overall direction of C++ with `<type_traits>` has eschewed the idea that there would ever be a used for `decltype( SomeType )` as a `constexpr object`-producing object. It could be safe to take `decltype ( type-id )` for C++ to just act as a placeholder for `type-id` to harmonize its version of `typeof` with C's, even if it might not provide any essential value.

But, we do **NOT** propose it here and consider it outside the scope of this proposal as there is no existing practice for `decltype ( type-id )`.

This proposal adds `typeof` and `typeof_unqual` to C++ to harmonize the 2 languages and close the loophole left from 20 years ago (for C++) and 30 years ago (from C), now that C has finally standardized its long-implemented keyword extensions.

§ 3. Specification

The specification is relative to the latest C++ Working Draft, [n4950].

§ 3.1. Language Wording

§ 3.1.1. Add to Table 5: Keywords ([tab:lex.key]) 2 new keywords

5.11 Keywords

[lex.key]

1 ...

alignas	constinit	false	public	true
alignof	const_cast	float	register	try
asm	continue	for	reinterpret_cast	typedef
auto	co_await	friend	requires	typeid
bool	co_return	goto	return	typename
break	co_yield	if	short	typeof
case	decltype	inline	signed	typeof_unqual
catch	default	int	sizeof	...
char	delete	long	static	
char8_t	do	mutable	static_assert	
char16_t	double	namespace	static_cast	
char32_t	dynamic_cast	new	struct	
class	else	noexcept	switch	
concept	enum	nullptr	template	
const	explicit	operator	this	
constexpr	export	private	thread_local	
constexpr	extern	protected	throw	

9.2.9.5 Decltype specifiers

[[dcl.type.decltype](#)]

decltype-specifier:

`decltype ( expression )`

`typeof ( expression )`

`typeof ( type-id )`

`typeof_unqual ( expression )`

`typeof_unqual ( type-id )`

- 1 For an expression *E*, the type denoted by `decltype(E)` is defined as follows:
 - (1.1) — if *E* is an unparenthesized *id-expression* naming a structured binding ([\[dcl.struct.bind\]](#)), `decltype(E)` is the referenced type as given in the specification of the structured binding declaration;
 - (1.2) — otherwise, if *E* is an unparenthesized *id-expression* naming a non-type template-parameter ([\[temp.param\]](#)), `decltype(E)` is the type of the template-parameter after performing any necessary type deduction ([\[dcl.spec.auto\]](#), [\[dcl.type.class.deduct\]](#));
 - (1.3) — otherwise, if *E* is an unparenthesized *id-expression* or an unparenthesized class member access ([\[expr.ref\]](#)), `decltype(E)` is the type of the entity named by *E*. If there is no such entity, the program is ill-formed;
 - (1.4) — otherwise, if *E* is an xvalue, `decltype(E)` is `T&&`, where *T* is the type of *E*;
 - (1.5) — otherwise, if *E* is an lvalue, `decltype(E)` is `T&`, where *T* is the type of *E*;
 - (1.6) — otherwise, `decltype(E)` is the type of *E*.

The operand of the `decltype`, `typeof`, or `typeof_unqual` specifier is an unevaluated operand.

...

- 3 ✨ For an expression *E*, the type denoted by `typeof(E)` is formed by removing reference qualifiers, if any, from the type denoted by `decltype(E)`. The type denoted by `typeof_unqual(E)` is formed by removing reference qualifiers, if any, as well as top-level cv-qualifiers from the type denoted by `decltype(E)`.

4 ✨ For a *type-id* *T*, the type denoted by `typeof(T)` is formed by removing reference qualifiers, if any, from *T*. The type denoted by `typeof_unqual(T)` is formed by removing reference qualifiers, if any, as well as top-level cv-qualifiers from *T*.

§ 3.1.3. Modify **decltype** mention in Template Deduction's General Clause [`temp.deduct.general`] to include `typeof` and `typeof_unqual`

13.10.3.1 General

[`temp.deduct.general`]

...

6 The deduction substitution loci are

(6.1) — the function type outside of the *noexcept-specifier*,

(6.2) — the *explicit-specifier*, and

(6.3) — the template parameter declarations.

The substitution occurs in all types and expressions that are used in the deduction substitution loci. The expressions include not only constant expressions such as those that appear in array bounds or as nontype template arguments but also general expressions (i.e., non-constant expressions) inside `sizeof`, `decltype`, `typeof`, `typeof_unqual`, and other contexts that allow non-constant expressions. The substitution proceeds in lexical order and stops when a condition that causes deduction to fail is encountered. If substitution into different declarations of the same function template would cause template instantiations to occur in a different order or not at all, the program is ill-formed; no diagnostic required.

...

§ References

§ Informative References

[N1978]

Jakko Järvi; Bjarne Stroustrup; Gabriel Dos Reis. *N1978 - Decltype (revision 5)*. April 24th, 2006. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2006/n1978.pdf>

[N2343]

Jakko Järvi; Bjarne Stroustrup; Gabriel Dos Reis. *N2343 - Decltype (revision 7)*. July 18th, 2007. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2343.pdf>

[N2927]

JeanHeyd Meneide; Shepherd's Oasis, LLC. *N2927 - Not-so-magic - typeof for C*. July 18th, 2007. URL: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2927.htm>

[N4950]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 10 May 2023. URL: <https://wg21.link/n4950>